

Concept for Designing an ICS Testbed from a Penetration Testing Perspective

Sebastian Kraust

Laboratory for Safe and Secure Systems (LaS³)

OTH Regensburg

93053 Regensburg, Germany

sebastian.kraust@oth-regensburg.de

Peter Heller

LaS³

OTH Regensburg

93053 Regensburg, Germany

peter2.heller@oth-regensburg.de

Jürgen Mottok

LaS³

OTH Regensburg

93053 Regensburg, Germany

juergen.mottok@oth-regensburg.de

Abstract—Industrial control systems are foundational to the operation of critical infrastructure and industrial processes. As such, they are prime targets for cyber attacks, necessitating a proper understanding of a system's attack surface. Cybersecurity testbeds provide a safe environment for exploring cyber attacks and their effects on systems and networks. However, devising a testbed can be a resource-intensive and time-consuming process. This is especially true from a penetration testing perspective, as most testbeds are not designed to support the exploration of chaining attacks against a wide variety of architectures, implementations, and software versions. This paper addresses this issue by proposing an iterative concept for creating flexible and scalable testbeds. It also presents lessons learned that influenced the concept and discusses and positions it in relation to current testbeds literature.

Index Terms—penetration testing, attack chaining, testbed concept

1. Introduction

In the field of cybersecurity, the ability to develop, validate and optimize algorithms in realistic environments is critical to protecting systems against threats efficiently and reliably. This is especially true for industrial control systems (ICS) in the operational technology (OT) domain, which are increasingly integrated with the conventional information technology (IT) infrastructure and therefore open up new attack surfaces. However, testing in live environments is not feasible due to the risk of compromising and damaging production equipment and possibly endangering human life and supply security. Testbeds offer a solution to these problems, as they serve as controlled environments that emulate ICS setups, enabling researchers and practitioners to safely study vulnerabilities, evaluate defensive mechanisms, and test responses to cyber threats. However, the majority of existing testbeds are designed from a defender's perspective, prioritizing the fidelity of the modeled processes over the exploration of adversarial strategies. This defensive viewpoint often limits the ability of these testbeds to examine more sophisticated attack scenarios, particularly those involving attack chaining or advanced adversarial tactics. Attack chaining is of particular interest, as the sequential exploitation of multiple vulnerabilities or misconfigurations to achieve a specific goal is underexplored. Among other things, this can be

attributed to the sheer number of possibilities and the accompanying complexity of managing a test environment.

This paper addresses this issue by proposing a novel concept for developing ICS testbeds that takes the point of view of an adversary (and, by extension, a penetration tester). By this we mean that the focus lies on its flexibility and bottom-up methodology. Rather than building a complex testbed upfront, we advocate starting with a small set-up with minimal functionality. After sufficient testing has been performed with the initial version, we can vary configurations, implementations, and versions and incrementally expand the testbed to incorporate additional components, capabilities, and vulnerabilities. This ensures that each layer of complexity is built upon a robust and well-understood base.

The contributions of this work can be categorized into two aspects:

C1: We propose an iterative testing concept to build a penetration testbed from scratch.

C2: We conduct an analysis of recent publications on cybersecurity and ICS testbeds and evaluate how testbeds are designed for industrial environments. We compare the different setups based on typical metrics found in the literature (architecture, design, scalability, use cases, reproducibility, and cost-effectiveness) and compare our penetration testing focused approach to other works.

The remainder of the paper is organized as follows: In Section 2, we review the literature on available ICS testbeds and testbed design concepts. Afterwards, we introduce an iterative testbed design concept in Section 3. We discuss and reflect on our approach in Section 4. We summarize our findings in Section 5 and provide an outlook for our upcoming work.

2. Related Work

In this section, we consult the relevant literature for the development and analysis of ICS testbeds, in order to highlight issues in the current research. To do so, we first have to examine the qualitative and quantitative metrics that can be used to compare and evaluate testbed publications. We then apply those to a selection of commonly referenced testbeds to identify gaps that limit their practical relevance. Afterwards, we present how other methodologies tackle the problem of deriving testbed designs from specific use cases.

2.1. Classification Metrics

In order to be able to compare testbed publications properly, we initially have to identify which key attributes can be used as metrics. Conti et al. provide a survey that covers the following relevant aspects [1]: First, it introduces the basic terminology for industrial control systems as well as the associated reference architecture, called the Purdue model after the IEC 62443 standard [2]. The latter divides ICS environments into a hierarchy of levels of increasing abstraction, ranging from physical processes on level 0 to enterprise planning on level 5. The so-called *Manufacturing Zone* (levels 0 to 3) encompasses all the systems of the OT part of the ICS, which are concerned with managing, controlling and monitoring the physical process. Devices at these levels commonly use application-specific industrial protocols with a varying number of security features. In contrast, *Enterprise Zone* (levels 4 and 5) comprises the IT section of the network with more traditional systems and protocols that are typical of an enterprise network. The two zones are usually separated by a demilitarized zone (DMZ), which manages and controls data exchanges between the IT and OT levels.

Second, the survey also provides an analysis of the protocols utilized in more than 80 publications related to testbeds and/or ICS datasets. One key finding is that most of the contributions focus on Modbus (39.5%), followed by DNP3 (12.9%) and EtherNet/IP (10.5%). This is particularly noteworthy compared to the actual market share of these protocols, where Modbus variants represent only 8% of newly installed nodes in industrial networks in 2024 [3]. Therefore, a worthwhile metric to consider is the number and type of protocols included in the testbed. It should also be noted that protocols with little or no coverage would benefit the most from the development of a new testbed.

Lastly, Conti et al. classify the selected publications based on a number of properties, which include their category (physical, virtual or hybrid), industrial sector (e. g. power grid), type of physics process (real or simulated), license, qualitative cost, and scope (purpose of the testbed, e. g. cyber security research). An interesting observation is that, even though the Purdue model is explained in detail beforehand, coverage of its levels is not used as a criterion.

Another similar survey, conducted by Pospisil et al., introduces a set of parameters that are used to compare 31 testbeds [4]. These parameters were compiled by the authors from several previous publications, which overlap in part with the previous study (although the terminology differs). Among the shared parameters are the testbed category, sector (called scenario), used protocols, and scope (called application scenario). In addition to these, other aspects are considered, such as the implemented levels of the Purdue model of the ISA-95 standard (which does not yet differentiate between levels 4 and 5 and combines them instead). However, this does not pose a problem for a comparison, as the authors limit themselves to levels 0 to 2. The next parameter is a component checklist that records the presence of essential industrial components, such as Programmable Logic Controllers (PLC), Human-Machine Interfaces (HMI), Master Terminal Units (MTU), Remote Terminal Units (RTU), Supervisory Control And Data Acquisition (SCADA) systems, engineering worksta-

tions, and historians. Lastly, the authors use the so-called characteristics, which is essentially a collection of miscellaneous focus points of the testbed. For example, check if the testbeds are concerned with some form of data analysis or generation, cybersecurity testing, cloud integration, and whether they were designed with scalability, extensibility, or flexibility in mind.

Although not an exhaustive list, the metrics presented provide sufficient insight for evaluating and comparing testbeds. In the following section, we use a selection of metrics to analyze popular ICS testbed publications.

2.2. Testbed Case Studies

In this section, we present a number of ICS testbeds, which we subsequently analyze with some of the metrics introduced in the previous section. We opted to focus on the following attributes: testbed category and associated cost, protocols used, and coverage of the Purdue model. Although the aforementioned surveys analyze coverage on a per-level basis, it is sufficient for our purposes to consider the broader segmentation into zones (enterprise, DMZ, and manufacturing). This granularity is sufficient to assess the coverage of the IT and OT domains. Additionally, we will pay special attention to whether the contribution explains how the respective testbed setups were derived. We summarize our findings in Table 1.

A popular testbed in the ICS domain is SWaT, which is a fully functional, small-scale version of a modern water treatment plant [5]. It implements an elaborate, six-stage process and was designed in collaboration with Singapore's nation-wide water utility company, to ensure realism. In addition, the authors also share their lessons learned, which raise some interesting points: firstly, due to the testbed being essentially a replica of a physical system, it scales poorly and therefore, access to it is limited. Secondly, the implemented industrial protocols (EtherNet/IP and CIP) were chosen on the advice of the testbed manufacturer without considering the sparse open-source tool support. Lastly, the proprietary nature of the used SCADA and historian software makes it very difficult to integrate, and especially the extraction of data from the databases is cumbersome and only manually possible.

The RICS-el testbed is a fully virtualized testbed to conduct experiments with SCADA systems in a power grid facility [6]. It features both an office IT and an OT network segment. The former contains a firewall, a DMZ providing endpoints for web protocols such as FTP, and a local office network with several work stations. The latter contains a second DMZ, a local OT network, as well as a process WAN and the process simulator. The network is populated by the so-called bots that exchange data between devices. The simulation is based on a training module provided by the SCADA manufacturer. Although the architecture is given, the testbed can not easily replicated due to the lack of detail, the use of proprietary software (i. e. the simulation), and the sheer complexity of the testbed. The design process is not described in depth; however, it is claimed that the design was created in collaboration with vendor experts.

Green et al. propose the Lancaster's testbed, a physical testbed that implements a generic ICS [7]. It has six *Manufacturing Zones* (containing equipment from levels

TABLE 1. COMPARISON OF TESTBED CHARACTERISTICS

Ref	Category	Cost	Protocols	Derivation	Manufacturing Zone	DMZ	Enterprise Zone
[5]	physical	high	EtherNet/IP CIP	no	✓		
[6]	virtual	low	IEC 60870-5-104 NTP, FTP, Web	no	✓	✓	✓
[7]	physical	high	KEPServerEX-supported IP	no	✓	✓	✓
[8]	virtual	low	IEC 60870-5-104, OPC UA	partial	✓		
[9]	hybrid	medium	s7comm, Profinet custom TCP	no	✓	✓	✓
[10]	hybrid	medium	EtherNet/IP, OPC, Modbus/TCP EtherCAT, DeviceNet	no	✓	✓	
[11]	virtual	low	Modbus, s7comm, OPC	no	✓		

0 to 3 in the Purdue model), a DMZ, and an *Enterprise Zone* (levels 4 and 5). The researchers also made an effort to make the testbed accessible from the internet. They also included elements to account for future extension and/or adjustments, for example, by including a management network, and a VMWare vSphere server for running desktop and server-based applications. Communication between nodes using legacy ICS protocols has been upgraded to IP to reduce configuration complexity and increase compatibility, but can be reverted back if necessary. The paper also comes with lessons learned, which notably includes the authors' approach to design. They argue that the choice of included devices and technologies should be market driven and that process diversity and complexity is not always crucial, as it allows for an easier reconfiguration of devices and hot-swap capabilities at the expense of exploiting physical process aspects. They also endorse the usage of VLANs and virtualization technologies, as it allows easier integration and scaling of testbed components. Ultimately, while the thought process behind the derivation of the architecture is not included in the paper, these insights should be taken into consideration for future designs.

Maynard et al. published an exemplary setup in the context of presenting a framework for creating, deploying, and managing SCADA testbeds [8]. The framework allows to standardize the description, building, and configuration of virtualized network nodes through the use of dedicated scripts. To demonstrate the advantages of this approach, the authors built a network with seven nodes, showcasing a simple metering application. Currently, the testbed implements two protocols, OPC UA and IEC 60870-5-104. However, other protocols can be added if necessary. The paper also provides a discussion of requirements that highlights additional key features that were taken into account during the design process. This includes seamless integration with physical networks and devices, integration of process simulators, and scalability. The complete derivation of the testbed layout is not discussed apart from the deployment, as the focus of the paper is on the framework. We account for this by denoting the derivation as 'partial' in Table 1.

The hybrid testbed published by Hui et al. is based on a nuclear reactor control system [9]. The focus is on providing realistic network interactions and data collection to investigate cyber attacks on physical processes. The testbed implements four interdependent sub-processes, each of which is controlled by its own PLC. Communication between them is performed using the s7comm,

Profinet, and custom TCP-based protocols. The comprehensive testbed includes IT and OT network sections, complete with a DMZ, firewalls, as well as physical actuators with dedicated control circuits. Even though the authors describe the processes and architecture in great detail, they touch only briefly on the design of the testbed, i. e. by citing best practices and reference architectures.

The NIST testbed is an exhaustive system created to evaluate the implementation of various security measures in industrial control systems by assessing their impact on overall network performance [10]. It consists of three separate industrial scenarios, in order to be able to emulate processes with different characteristics (such as fast and slow dynamics). The scenarios are the Tennessee Eastman process (TE for short, a chemical reactor process), a robotic assembly, and an intelligent transportation system together with a pipeline network. The last scenario was not finalized at publication time and only a rough outline is given. Each process is placed in a separate enclave, each with their own set of chosen protocols, architecture, and devices. The testbeds also allow for data collection, packet tampering, and management over a fourth enclave. Although the authors lay out their reasoning for choosing the scenarios and describe the processes in great detail, the derivation of the concrete architecture is not discussed.

The Virtual Tennessee-Eastman Testbed (VTET) is a virtual testbed that employs Matlab to simulate the underlying process [11]. Despite the name, it is also possible to replace the PLC simulation with real hardware, thus effectively changing to a hybrid mode. The architecture consists of four devices: a PC simulating the TE process and for network communication, respectively, a PC simulating the virtual PLC, and the physical PLC. The testbed utilizes three major industrial protocols (OPC, s7comm, Modbus), and has a strong focus on attacking the industrial process itself. This is demonstrated by presenting five attack scenarios, complete with diagrams showing process parameters over time. Due to the physical devices and toolboxes used, attacks are limited to the specific manufacturer. Again, this paper also does not present any derivation of the architecture, reasoning behind the choice of devices, software, etc.

We will discuss the results later in Section 4 in detail. However, it is clear that the testbed publications often omit how they derive their design. In the following section, we will take a look at existing methodologies that can be used for testbed development.

2.3. Testbed Development

In this section, we discuss proposals on how to systematize the process of developing testbeds in the literature.

We have already briefly touched on the framework introduced by Maynard et al. [8]. It is a collection of scripts that deploy and configure virtual machines (Oracle VirtualBox by default); however, other technologies, such as containerization, can be used instead. The deployment process has three stages: a *Compilation Stage* where the required software is compiled for all nodes, followed by an *Orchestration Stage*, which uses two sets of scripts named *Operation Profile* and *Configuration Profile*. The former defines the deployment of nodes, simulators, and the configuration of the networks. The latter defines how a certain node is configured in order to act as a certain kind of system (e. g. a HMI). The authors also propose to collect and share these profiles publicly, in order to facilitate the deployment of testbeds. Based on the operational profile, each node gets its respective configuration profile and gets assigned an address. During boot-up, the nodes install the necessary packages according to their configuration profile. Once configured, the *Operation Stage* is reached, the configured communication starts, and the operator can interact with the nodes. Although this approach does not address how to design the architecture of a testbed, it shows how to standardize and simplify the deployment after the initial design.

An educational testbed in conjunction with a new design methodology is suggested by Frank et al. [12]. This so-called *Design Life Cycle Model* addresses the lack of a proper derivation process for a testbed design if certain security challenges must be evaluated within a certain environment. It has two components: the first part contains the management of a test environment and all related tasks, such as selecting a network architecture, placement of application, etc. The second part is concerned with the aforementioned challenges on top of the current environment. This includes the selection of specific software, applications, and their respective versions. The testbed description itself is a proof-of-concept of their methodology and focuses on a cloud-based approach that enables a lecturer to provide students remotely with a variety of security challenges based on their skill level. This approach tackles the design from a goal-oriented perspective, as the first task is to identify which kind of tests shall be conducted (attack web applications, reverse engineering software, etc.). Based on this, the necessary components and features of the testbed are chosen and translated into requirements. Even though this implicitly influences the architecture, this still leaves a lot of room for interpretation.

The next section introduces our own concept, which is then positioned and analyzed within the presented literature in Section 4.

3. A Penetration Testing driven Approach to System Modeling

The suggested approach aims to formalize the process of creating modular and scalable penetration testing environments to advance cybersecurity research. These

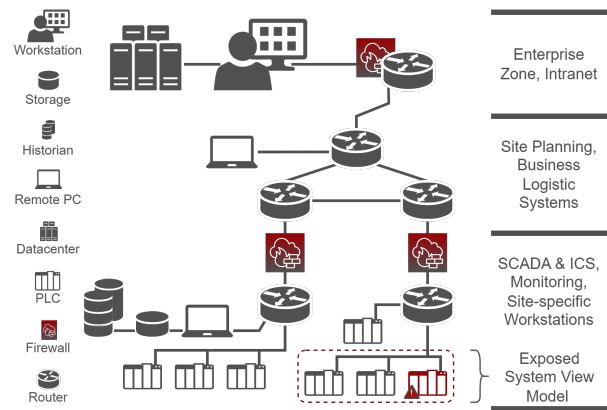


Figure 1. The network architecture of an exemplary system model that covers different levels of the automation hierarchy (See Section 2.1). The exposed system view model is indicated by the red dashed frame and shows which devices are initially visible to an adversary. The red icon therein marks one possible entry point for an adversary, which attempts to breach the security boundaries. The entirety of the network (including the system view model) is a part of the static system model. Traversing the perimeter of the exposed system view model indicates a breach and therefore, a successful attack.

environments are crucial for safely developing and testing security measures, including activities such as the creation of intelligent algorithms for intrusion detection, vulnerability analysis, and data generation for AI-based models. In this section, we first present the overall concept and how it can be used to iteratively derive systems for penetration tests. The second part then provides an in-depth explanation on how to plan and execute tests within such a system.

3.1. Deriving a System for Penetration Tests

To illustrate the core concept, we use the representative industrial control system shown in Figure 1. The network architecture includes an OT segment with generic control devices on the lower levels and an IT segment for enterprise systems on the upper levels. The process is iterative in nature and is divided into two stages: a static system definition and a dynamic testing cycle. It is visualized in Figure 2.

Stage I, called the *Static System Model (SSM) Definition*, describes the modeled system and contains information about the adversary and his goals. It is subdivided into two steps. Step 1 defines the model attributes and captures the overall functionality of the test system for the current iteration. This includes the architecture and devices, the communication relationships between them, a number of external entry points or service providers (if needed), and the services and applications running on the devices. The exact configurations of the devices and endpoints are not specified here, but during the dynamic testing phase of Stage II to enable/disable certain vulnerabilities as necessary for the tests. The SSM can be expanded and/or changed once sufficient testing has been performed, which starts a new iteration.

Step 2 contains the definition of the attacker, their respective attack goals, and a specific subset of the SSM network that is initially visible and accessible to the adversary. This restricted view is called the *System View*

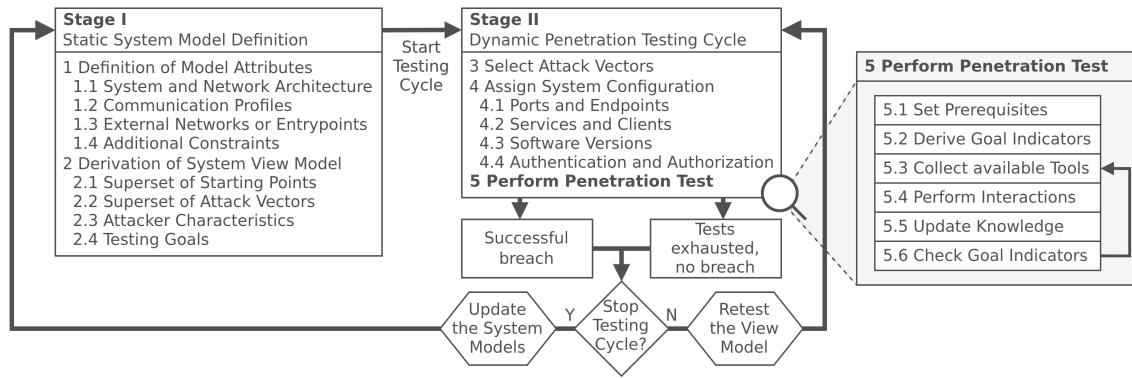


Figure 2. The proposed two-stage process for deriving a testbed. The first stage defines a static system model, which includes system boundaries, attacker definition and the testing goals. As part of the second stage, the attacker is given a view of the system architecture to interact with the virtual environment. The stages are iteratively refined and updated to gradually build more complex setups. The penetration testing procedure described in Step 5 is shown in-depth on the right.

Model (SVM), and is depicted as a dashed box in Figure 1. The dividing line between this view and the rest of the network defines the security boundaries. A violation of these boundaries indicates a successful breach. Once a SVM has been selected, a testing cycle starts. During a cycle, a series of tests is conducted during which the attack surface is varied by altering the placement of the attacker (the initial starting point), by using variations of initial access on the SVM, and also by using known vulnerabilities or configuration errors. If the attacker is able to achieve the defined goal (e. g. move outside the system boundaries or impact system performance), we can use captured interactions to analyze the theoretical impact and possibly update the system models before performing additional tests. By doing this, we can test the effectiveness of different mitigation strategies or explore how a chain of vulnerabilities can be exploited. Once a testing cycle for a specific static system model has been completed, it can be modified for the next iteration by adding complexity or introducing mitigations for existing vulnerabilities. This allows us to start out with a relatively small setup with a limited number of devices to explore specific vulnerabilities or configuration errors. We can then gradually extend the model to investigate more complex interactions between systems and, more importantly, defense-in-depth strategies for industrial systems.

During **Stage II**, called the *Dynamic Penetration Testing Cycle*, several penetration tests are executed by defining and varying attack vectors, initial access points, and system configurations, followed by an impact analysis. Once again, the activities in this step can be divided into several steps, starting with Step 3. Therein, we choose the exact placement of the attacker within the initially accessible SVM. In Step 4, the exact system configurations are finalized for all components. For each testing cycle, a number of specific vulnerabilities and configurations are selected to evaluate whether an attacker is able to compromise the system, i. e. by impacting availability, performance or through lateral movement. Crossing the aforementioned security boundaries represents a violation of the system integrity. Therefore, a test is considered successful if a security boundary between the initial SVM and the rest of the network is breached. Under the cir-

cumstances of a specific test, an adversary can explore different attacks to break through until all options are exhausted. Lastly, Step 5 describes the design and actual execution of penetration tests, for which the parameters of the testing environment are fixed. We provide an in-depth explanation of this step in Section 3.2.

After performing sufficient testing for the current cycle, we can analyze which attacks were able to achieve their goals under the given configuration options and attack vectors. This information can be used to identify flaws in the system's security concept. After the evaluations, there are two possible options on how to proceed: the first is to reuse the current models and run additional tests for different attack vectors and system configurations. The other option is to modify the static model or the SVM, to begin a new testing cycle within the new environment. By doing it this way, the tester can ramp up the complexity of the setup and therefore the attacks while building on a robust basis of understanding. Using a static system model in conjunction with a separate view model facilitates the exploration of different combinations of configuration options and attack vectors. This allows for the modeling and assessment of complex attack chains.

3.2. Planning and Performing a Penetration Test

We use the information provided by the MITRE ATT&CK[®] [13] framework as an example of how to model and design a penetration test for an abstract system. ATT&CK is a knowledge base that defines tactics and techniques for the development of threat models with respect to certain system domains (Enterprise, ICS and Mobile). This is intended to serve as one possible example of how our concept can be used. For a practical application, the static system model, the system view model, and any configurations for the dynamic penetration testing environment need to be considered for further steps. To successfully execute a penetration test in a simulated environment, we need to perform a series of repetitive steps to interact with the system models.

The testing activity for penetration tests and vulnerability assessment can be broken down into six basic steps, which are depicted in the in-depth view of Step 5 in Figure 2. The first two steps (5.1 and 5.2) are preparatory

steps required to define the starting point and the final goal of the penetration testing process. The cycle of steps 5.3 to 5.6 abstracts the interaction between an attacker or tester with the testbed system. First, the available tools are enumerated for interacting with the testbed. Second, a specific interaction is chosen and executed. Based on the response of the system, knowledge about the current state and architecture of the system are evaluated and updated. Then we determine whether the interaction was successful with respect to the attacker being able to impact our system or move outside of the chosen SVM.

Our **Prerequisites** define the properties of the subsystem that is exposed to the attacker, as well as the entry point of the attacker inside the system. This step is used to choose the initial starting point for a penetration test, which means that we determine the exact placement of the attacker. We assume that an attacker has already gained access to a specific subsystem or network component inside the SVM, and define the exact point of entry as part of our prerequisites. As a guideline for further planning or refining this part of the test, MITRE techniques from the **Initial Access** and **Credential Access** tactics can be used. From there we shift our focus to the different possibilities an attacker might implement and simulate different attack paths during the next steps.

The steps **Derive Goal Indicators** and **Check Goal Indicators** are related to the monitoring of the agent (tester) or attacker with regard to lateral movement and elevation of privileges. As our tests progress, we need to determine if an attack based on a specific interaction was successful, how the system is impacted, or generally speaking our definition of done. To determine different possible goals and suitable indicators of a successful test, we can generally cover two main aspects: Lateral Movement and System Performance Impact. This indicates that an attacker has either a) the means to actively move beyond security boundaries of the chosen SVM, or b) can impact the operational status (e. g. availability, performance metrics, deadlines) of specific assets. The boundary introduced between the SVM and SSM serves as a metric to determine whether lateral movement between both systems is possible. The aspects of performance impact or operational status must be determined by the tester on the basis of the requirements of the specific testbed. Typically techniques from the groups Execution, Defense Evasion, Impact or Lateral Movement are suitable for these goals and can be used to implement specific detection methods inside a testbed.

MITRE ATT&CK currently offers 14 available techniques describing the impact of attacks on enterprise and ICS resources. These techniques can generally be summarized as manipulation, interruption, or destruction of data and systems. Certain conditions need to be satisfied to indicate a successful test for these goals, such as specific levels of privileges on critical devices or device configurations that need to be accessed/manipulated by the adversary. These indicators are specific to an industrial setting and serve as a reference to define when an attack has been successful. By gaining a certain level of privileges or by crossing the boundaries of the view model, we can use the captured interaction of the agent or an attacker to provide an impact assessment.

Step 5.3, **Collection of available tools**, indicates how

an attacker or tester could enumerate available tools and services that would allow him to interact with the system. This step of the testing process is related to the enumeration of locally available tools and executables, but also to the collection or installation of remotely available tools in any meaningful way. A typical interaction in this regard could be installation of scripts, vulnerable files, or locally compiling exploitable binaries to target specific devices or vulnerabilities before actually executing the attack.

Steps 5.4 and 5.5 (**Perform Interactions** and **Update Knowledge**) are related to interacting with systems inside the testbed to gain (unauthorized) access or impact services. This step describes the execution of a targeted interaction with the motivation to gain specific information about a system or the targeting of specific vulnerabilities, misconfigurations, or entry points. To this end, the MITRE ATT&CK techniques and sub-techniques are used to assemble a selection of tools that facilitate interaction with the targeted system. Through interaction, the adversary is able to gain feedback from the system and thereby infer knowledge about the current state and architecture. The following tactics provide a comprehensive reference to derive a finite set of possible interactions: *Reconnaissance*, *Collection*, *Discovery*, *Execution*, *Credential Access*, *Defense Evasion*, *Privilege Escalation*, *Inhibit Response Function*, *Impair Process Control*. The agent must select the most appropriate tool from the available tactics to advance within the SVM, thus extending their access to system resources.

The following section positions our approach in relation to the previously presented literature. We also highlight the research gaps and discuss how they are addressed by the concept.

4. Discussion

In Section 2, we have shown that holistic testbed design methodologies are a neglected topic in the literature. In the following we discuss how our concept addresses these shortcomings.

The presented concept, in essence, aims to systematize the testbed development process. We have shown that this topic is neglected in the current literature. Although understanding the reasoning behind design choices is valuable, we understand that the authors of testbed publications primarily want to present the finished product and, therefore, focus on explaining architectures, processes, and utilized hardware and software. However, it is remarkable how few methodologies exist that address the design process itself. We have presented concepts for the deployment [8] and requirements engineering [12] of ICS testbeds, but these address only a specific part of the overall process or are very abstract in nature. Hence, we devised a concept that assumes starting from scratch. The iterative bottom-up approach ensures that the developers do not get overwhelmed with complexity, but gradually build towards a more sophisticated system. Our concept was also inspired by keyword-driven testing [14], as defining tests on multiple levels of abstraction is beneficial for the dynamic penetration testing procedure. Hence, test goals can first be defined independently from the underlying implementation, before they are broken down into concrete steps for a specific variant as part of the penetration

testing process. By incorporating these aspects, we think that the general concept is universally applicable to any penetration testing-focused testbed.

The review of common ICS testbeds revealed that they are usually designed from a defender's perspective. By this we mean that the testbed is either modeled after a real system or a standard reference model, and not intended to be modified in any major way after it has been set up. This is especially true for complex physical testbeds such as SWaT [5], as installation often requires a significant investment in time and resources. This contradicts directly the typical modus operandi of a penetration tester, which is characterized by quickly alternating between software and hardware products, versions, patches, and configurations to identify security issues. These static setups often lack adaptability, configurability, and scalability, all of which delay testbed updates and could cause security issues to go unnoticed. In addition, most testbeds are process-oriented, which means that they replicate an intricate process which is then attacked (e. g. the Tennessee Eastman process as shown in [11]). However, disturbing a process is not the primary goal of our penetration tests, as they effectively test the process simulation, and not vulnerabilities of the network and their interactions. Disturbing the process is rather a result of hijacking devices and changing process parameters, and as such, our focus is network-oriented. Using a comparatively simpler process simulation, individual components can be easily swapped out, especially when packaged as a container application. This quick varying of the attack surface allows us to rapidly configure the system and tweak parts of the architecture to build and test complex attack chains. By adopting an adversary's perspective, we hope to facilitate the development of testbeds for penetration testers and, therefore, improve overall security.

Previously, we presented a comparison of testbeds and summarized our findings in Table 1. Although approximately two-thirds of all testbeds are made up of a small set of protocols (Modbus, DNP3, EtherNet/IP, s7comm), only a small number use more modern protocols like OPC UA [1]. For this specific protocol, this can be attributed in part to the relative newness of the most recent versions. An additional factor could be that OPC UA's contemporary characteristics prevent it from being seen as a traditional industrial protocol, resulting in its limited awareness among many researchers. Another aspect is that the survey by Conti et al. does not differentiate between OPC derivatives and instead groups them together. Even with this broad categorization, they represent only 2.4% of all testbeds. On closer inspection, only the testbed by Maynard et al. utilizes OPC UA [8]. It is, however, noteworthy that it is utilized solely to connect a single device (the data historian) to the remaining network. Based on the reviewed information, it appears that no existing testbed currently prioritizes OPC UA as its main protocol. Given the growing importance of OPC UA in industrial systems, the creation of a testbed with this relatively underutilized protocol and its security features appears to be valuable.

Another observation is that testbeds that model devices in both the manufacturing and enterprise zones usually separate them by using a DMZ. The purpose of a DMZ is to isolate the OT from the IT portion of the network. In this way, access to sensible OT equipment can be

restricted, while certain production data can be made available through servers placed within the DMZ [6], [9]. As a security element, it is an interesting target for penetration tests that aim to break through. However, most publications do not model the enterprise part at all [5], [8], [11], or lack a description of the data flows between the zones [9], [10]. This is especially interesting in the context of protocols like OPC UA, which implements a service-oriented architecture. That means that devices communicate using standardized services instead of just exchanging raw data. Examples of these include discovery and authentication services that require bidirectional interactions between different endpoints and devices. Particularly since the only testbed that includes OPC UA models only the OT part of the network, it would be valuable to explore how an attack could traverse the DMZ, either from OT to IT or vice versa.

Finally, it intuitively makes sense that physical testbeds can replicate a real system with higher fidelity than virtual or hybrid setups. However, this causes these setups to be much more expensive, and often entails limited access to the testbed [5]. All of these factors contribute to poor reproducibility. As a result, we think that it is acceptable for testbeds with a penetration testing focus to forgo some of their fidelity in favor of accessibility. This goes hand in hand with the aforementioned need for flexibility, as technologies such as containerization offer an inexpensive solution for packaging and sharing software modules, which in turn enables wider adoption in the research community. Also, swapping out containers is fast and easy and even allows a testbed to switch to hybrid mode, as a container can simply be swapped out for a real device.

Going forward our goal is to use the presented concept in conjunction with these insights to create an OPC UA testbed. In the next section, we summarize the findings and contributions of this paper and provide an outlook for future works.

5. Conclusion and Outlook

In this paper, we introduced an iterative two-stage concept that allows us to gradually build up a testbed for penetration testing. It works by first defining a system model and then selecting a subset of it. Within this perimeter, different tests can be modeled by placing an adversary at different starting points and letting him interact and move in this restricted environment, which we call the SVM. We show how the MITRE ATT&CK® framework can be leveraged to derive specific goals and techniques for such tests.

The second contribution of this work is a literature analysis of current ICS testbed publications. The key findings are as follows: First, we discovered that the testbed design process in this domain is often neglected in the current literature. Second, most available ICS testbeds are process-oriented and therefore not suitable for testing and evaluating attack chains within a penetration testing context. Lastly, we introduced metrics to compare available testbeds and highlighted how future testbeds could incorporate these findings into their designs.

Looking ahead, future work aims to provide a proof-of-concept of our approach by using it for developing an

OPC UA testbed. We plan on open-sourcing the source code. This testbed will be used in our research context to generate training data for AI-based intrusion detection systems, as well as an agent for penetration testing automation.

Acknowledgment

The presented work is part of the research project *KRITIS Scalable Safe and Secure Modules* (KRITIS³M), which is funded by the Project Management Jülich (PtJ) and the German Federal Ministry for Economic Affairs and Climate Action (BMWK) under funding code 03EI6089A.

References

- [1] M. Conti *et al.*, “A Survey on Industrial Control System Testbeds and Datasets for Security Research,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2248–2294, 2021, ISSN: 1553-877X. DOI: 10.1109/COMST.2021.3094360.
- [2] IEC, 62443 - *Security for industrial automation and control systems*, 2019.
- [3] M. Jansson, *Annual analysis reveals steady growth in industrial network market*, <https://www.hms-networks.com/news/news-details/17-06-2024-annual-analysis-reveals-steady-growth-in-industrial-network-market>, Jan. 2025. (visited on 01/01/2025).
- [4] O. Pospisil *et al.*, “Application Perspective on Cybersecurity Testbed for Industrial Control Systems,” *Sensors*, vol. 21, no. 23, p. 8119, Jan. 2021, ISSN: 1424-8220. DOI: 10.3390/s21238119.
- [5] A. P. Mathur *et al.*, “SWaT: A water treatment testbed for research and training on ICS security,” in *2016 International Workshop on Cyber-physical Systems for Smart Water Networks (CySWater)*, Apr. 2016, pp. 31–36. DOI: 10.1109/CySWater.2016.7469060.
- [6] M. Almgren *et al.*, “RICS-el: Building a national testbed for research and training on SCADA security (short paper),” in *Critical Information Infrastructures Security*, Cham: Springer International Publishing, 2019, pp. 219–225, ISBN: 978-3-030-05849-4. DOI: 10.1007/978-3-030-05849-4_17.
- [7] B. Green *et al.*, “Pains, gains and PLCs: Ten lessons from building an industrial control systems testbed for security research,” in *10th USENIX Workshop on Cyber Security Experimentation and Test (CSET 17)*, Vancouver, BC: USENIX Association, Aug. 2017.
- [8] P. Maynard *et al.*, “An Open Framework for Deploying Experimental SCADA Testbed Networks,” in *5th International Symposium for ICS & SCADA Cyber Security Research 2018: Proceedings*, Hamburg, Germany: BCS, Aug. 2018. DOI: 10.14236/ewic/ICS2018.11.
- [9] H. Hui *et al.*, “ICS Interaction Testbed: A Platform for Cyber-Physical Security Research,” in *Proceedings of the 6th International Symposium for ICS & SCADA Cyber Security Research 2019*, BCS Learning & Development, Sep. 2019. DOI: 10.14236/ewic/icscsr19.12.
- [10] R. Candell Jr. *et al.*, “An Industrial Control System Cybersecurity Performance Testbed,” National Institute of Standards and Technology, Tech. Rep. NIST IR 8089, Nov. 2015. DOI: 10.6028/NIST.IR.8089. (visited on 12/29/2024).
- [11] Y. Xie *et al.*, “VTET: A Virtual Industrial Control System Testbed for Cyber Security Research,” in *2018 Third International Conference on Security of Smart Cities, Industrial Control System and Communications (SSIC)*, Oct. 2018, pp. 1–7. DOI: 10.1109/SSIC.2018.8556732.
- [12] M. Frank *et al.*, “Design considerations for cyber security testbeds: A case study on a cyber security testbed for education,” in *2017 IEEE 15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*, Nov. 2017, pp. 38–46. DOI: 10.1109/DASC-PiCom-DataCom-CyberSciTec.2017.23.
- [13] The MITRE Corporation, *MITRE ATT&CK® framework*, Jan. 2025. [Online]. Available: <https://attack.mitre.org/> (visited on 01/16/2025).
- [14] ISO/IEC/IEEE, 29119-5:2016 - *Software and Systems Engineering – Software Testing – Part 5: Keyword-Driven Testing*, 2016.